

Enterprise Cybersecurity & Infrastructure Security Strategy — Dyaboo Hybrid-Cloud ERP (.NET 8 / Next.js / PostgreSQL on On-Prem + AWS)

TL;DR

- Dyaboo should adopt an **identity-centric, Zero-Trust hybrid architecture**: keep on-premises Active Directory as the source of truth, federate to AWS via IAM Identity Center (SAML 2.0 + SCIM or direct AD Connector), enforce phishing-resistant MFA, and front the public-facing PLM/SAG cloud components with AWS WAF while keeping the latency-sensitive WMS on-prem behind private connectivity.
- Use **AWS-native managed services first** to minimize operational overhead for a fast-growing mid-size company (IAM Identity Center, KMS with customer-managed keys, RDS for PostgreSQL with encryption, GuardDuty/Security Hub, WAF, Secrets Manager, Session Manager), and adopt **portable open-source alternatives** (Keycloak, HashiCorp Vault, Wazuh, Trivy/Semgrep, OWASP ZAP) where portability, cost, or on-prem reach justify them.
- Prioritize in three phases: **(1)** foundational identity, encryption, connectivity, and logging; **(2)** DevSecOps

pipeline gates, WAF tuning, secrets management, SIEM correlation; (3) advanced Zero-Trust microsegmentation, mTLS, PAM maturity, and continuous compliance evidence for SOC 2 / ISO 27001.

Key Findings

- AWS's own guidance is that **IAM Identity Center is the recommended approach for new workforce-SSO deployments**, while SAML/ADFS federation remains valid for legacy constraints; if Dyaboo already runs AD, Identity Center can connect directly via AWS Directory Service / AD Connector, avoiding SAML+SCIM setup entirely while still issuing only short-lived STS credentials (no long-term access keys). (AWS)
- For the .NET 8 backend, Microsoft recommends **Microsoft.Identity.Web** layered over JWT bearer authentication; tokens are validated locally against the issuer's JWKS endpoint, so the API never calls the IdP per-request. The same stock `Microsoft.AspNetCore.Authentication.JwtBearer` validates Keycloak tokens via OIDC discovery — no vendor lock-in in code.
- **Direct Connect is the right primary link** for the on-prem ↔ AWS data path (predictable latency, MACsec up to 100 Gbps, regulatory private connectivity), with a **Site-to-Site VPN (two tunnels, BGP, terminating on a Transit Gateway)** as automatic failover. A pure VPN is

acceptable as a starting point for a fast-growing company until traffic/latency justify Direct Connect's cost and lead time.

- RDS for PostgreSQL encryption must be decided **before** instance creation (the KMS key cannot be changed afterward); use **customer-managed keys (CMKs)** with envelope encryption and automatic rotation, and layer **column/field-level encryption** (pgcrypto, or application-side) for the most sensitive IP and PII fields.
- A practical DevSecOps stack for this .NET/TypeScript estate: **SAST** (CodeQL via GitHub Advanced Security, or Semgrep/SonarQube), **SCA** (Dependabot + Trivy/Snyk), **secrets** (Gitleaks/TruffleHog + push protection), **containers/IaC** (Trivy + Checkov), **DAST** (OWASP ZAP), plus **SBOM + signing** (Syft + Cosign/Sigstore keyless).
- A hybrid SIEM is essential because **AWS Security Hub/GuardDuty cannot see on-prem servers, endpoints, or warehouse workloads**; the pragmatic answer is to run AWS-native detection for the cloud and forward findings into a cross-platform SIEM (Wazuh open-source, or Splunk/Microsoft Sentinel) that also ingests the on-prem WMS, with agents providing file-integrity and endpoint telemetry.

Details

Assumptions & Architecture Baseline

Per the brief, the following defaults apply unless noted: AWS is the cloud, leaning AWS-native; on-prem AD is the identity source of truth federated to the cloud; compliance posture targets SOC 2 and ISO 27001 alignment with GDPR considerations for supplier/customer personal data; **WMS runs on-premises** for low-latency warehouse operations while **PLM and SAG are public-facing cloud components**. I treat "SAG" as a public-facing ERP module (the brief does not expand the acronym); its controls mirror PLM's as an internet-exposed .NET 8 API + Next.js front end. The recommended account topology is a small **AWS Organization** with separate accounts for production, non-production, security/log-archive, and shared services/networking — this is the natural unit of blast-radius isolation and maps cleanly to SOC 2/ISO evidence scoping.

1. Identity & Access Management (IAM)

Federation backbone. Keep on-prem AD authoritative. Two supported patterns:

- **Direct AD connection** via AWS Directory Service / AD Connector to IAM Identity Center — Identity Center reads users/groups natively from AD, no SAML or SCIM

configuration, and you choose which AD groups to surface. Trade-off: you are tied to AD's structure and cannot mix multiple IdPs or use SAML attribute mapping.

- **SAML 2.0 + SCIM** when you front AD with an IdP (ADFS, or Entra ID if Dyaboo adopts Microsoft 365). SCIM keeps Identity Center's directory in sync — user disabled in the IdP is disabled in Identity Center, solving the "former employee still has AWS access" problem. Certificate rotation on the IdP signing cert is the most common cause of SAML outages, so monitor expiry.

All federated console/CLI access resolves to **IAM roles assumed via short-lived STS credentials** — never create IAM users for employees. Each role represents a job function, not an individual, and roles should not be reused across unrelated teams (it breaks audit and incident response). MFA must be enforced at the IdP **before** the SAML assertion is issued, because AWS trusts the IdP's authentication strength.

 

Workforce vs. machine/warehouse identities.

- *Internal warehouse staff (on-prem WMS):* authenticate against AD directly (Kerberos/NTLM on the LAN); the WMS .NET API validates AD-issued tokens or uses Windows Integrated Auth. Keep this path local so a WAN outage never stops the warehouse floor.

- *Remote administrators:* IAM Identity Center SSO portal → short-lived role sessions, gated by MFA and conditional access; shorten session duration to reduce stolen-token blast radius.
- *Cloud services / machine identities:* use **IAM roles for service-to-service** (EC2 instance profiles, ECS task roles, IRSA for EKS) so no static keys exist. For workloads that must hold a secret (e.g., the .NET API's DB credential), pull it at runtime from Secrets Manager or Vault.
- *Service-to-service auth across the hybrid boundary:* OAuth2 client-credentials grant (machine tokens) issued by the IdP, or mTLS with workload certificates (see §3).

.NET 8 backend auth. Register the API in the IdP (Entra ID, Cognito, or Keycloak), expose scopes/roles, and wire JWT bearer validation:

- **With Entra ID:**

```
AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddMicrosoftIdentityWebApi(Configuration.GetSection("AzureAd"))
```

(the `Microsoft.Identity.Web` package). The middleware downloads Entra's JWKS keys and validates signature, issuer (`iss`/`tid`), audience (`aud` = API client ID), and expiry. Enforce scopes via the `scp` claim (delegated) or

`roles` claim (application) and `VerifyUserHasAnyAcceptedScope`.

- With **Keycloak (open-source/portable)**: stock `Microsoft.AspNetCore.Authentication.JwtBearer` with `Authority = https://<kc>/realms/<realm>`, `MetadataAddress` = the realm's `.well-known/openid-configuration`, and `Audience` matching the token's `aud`. Keycloak nests roles under `realm_access.roles` and `resource_access.{client}.roles`, so add an `IClaimsTransformation` to flatten them into `ClaimTypes.Role` before `[Authorize(Roles=...)]` works.
- **RBAC model**: map AD groups → IdP roles → ASP.NET Core authorization policies. Prefer policy-based authorization (`RequireClaim`/`RequireAssertion`) over scattered role checks.

Next.js frontend auth. Use **Auth.js (NextAuth)** with the IdP's OIDC provider (Entra ID, Cognito, or the built-in **Keycloak provider**), using **Authorization Code flow with PKCE** (automatic for OIDC providers). Critical hardening:

- Store sessions in **HttpOnly, Secure, SameSite cookies** (`__Secure-`/`__Host-` prefixes) — never in `localStorage`, which is exposed to XSS. Auth.js sets signed/encrypted cookies, CSRF tokens, and clickjacking defenses by default. `Createbytes`

- Implement **refresh-token rotation** with replay detection; keep `AUTH_SECRET` in a secret store; do middleware-based route protection but treat edge "optimistic" checks as UI-only — enforce authorization server-side (Server Components/route handlers) for anything touching data. `Next.js`
- Forward the IdP access token to the .NET API in the `Authorization: Bearer` header (capture `account.access_token` in the `jwt`/`session` callbacks). Note Auth.js (formerly NextAuth) has seen maintenance/version churn (v5/Auth.js); for teams wanting less "assemble-it-yourself" risk, a managed option (WorkOS, Auth0, Cognito Hosted UI) is a defensible alternative. `WorkOS`

PAM, MFA, conditional access.

- Replace SSH bastions with **AWS Systems Manager Session Manager** — no open inbound ports, no long-lived SSH keys, IAM-gated, and every session logged to S3/CloudWatch. Session Manager also manages **on-prem servers** (advanced-instances tier), giving one auditable access plane across the hybrid estate. State Street publicly documented building their PAM on Session Manager precisely to avoid costly commercial PAM; in an AWS Cloud Operations Blog guest post, State Street's Herman Lee (Cloud Solution Architect, VP) and Nauman Noor (Managing Director) note that

"the team looked at commercial PAM offerings and found that they are often costly and involve complex account and password management of the privileged accounts." ([DEV Community...](#))

- For **just-in-time privileged elevation**, use the AWS sample JIT solution (Systems Manager Change Manager + IAM Identity Center temporary assignments) or **TEAM (Temporary Elevated Access Management)** for Identity Center; if using Entra, **Entra PIM** integrates with Identity Center for time-bound, approval-gated, MFA-enforced group activation. ([GitHub](#))
([AWS](#))
- For deeper enterprise PAM (credential vaulting, session recording for third-party vendors), **CyberArk** or **WALLIX Bastion** are available on AWS Marketplace, but for a mid-size company Session Manager + JIT covers most needs at far lower cost.
- Enforce **phishing-resistant MFA** (FIDO2/WebAuthn or at least TOTP) at the IdP; Identity Center supports WebAuthn and TOTP when AD is connected directly.

([AWS](#))

Keycloak vs. Cognito decision. For Dyaboo's *on-prem-AD-as-source-of-truth* hybrid posture, **Keycloak is the stronger open-source choice**: it has first-class native LDAP/AD User Federation (READ_ONLY edit mode recommended, ([group-ldap-mapper](#)) with

`LOAD_GROUPS_BY_MEMBER_ATTRIBUTE_RECURSIVELY` for nested AD groups, optional Kerberos/SPNEGO desktop SSO, and the AD-specific `msad-user-account-control-mapper`), whereas **Cognito has no direct LDAP/AD integration** and needs AWS Directory Service or a Lambda bridge. Keycloak is OIDC/OAuth2/SAML-compliant, supports RFC 8693 token exchange (standard V2 enabled by default), and scales horizontally (stateless Quarkus nodes + PostgreSQL + Infinispan). Keycloak's own benchmark (published Oct 1, 2025, author Pedro Ruivo) found that "Keycloak can handle significant loads while scaling almost linearly to 12,000 requests per second" — the top tier was 2,000 logins/sec plus 10,000 token refreshes/sec on a db.r8g.16xlarge Aurora PostgreSQL 17.5 instance with 100,000 users. The trade-off is **operational overhead** — you own patching, clustering (JDBC_PING/JGroups discovery), backups, and HA — so if Dyaboo wants minimal ops burden and is comfortable with AWS coupling, Cognito or a managed-Keycloak vendor (Red Hat build of Keycloak, Skycloak, Phase Two) is reasonable. Cognito password hashes cannot be exported, creating migration lock-in.

2. Network Security & Architecture (Hybrid Connectivity)

Primary link: Direct Connect; failover: Site-to-Site VPN.

- **Direct Connect** gives dedicated, private, predictable-latency, high-bandwidth (up to 100 Gbps) connectivity, helps meet regulatory private-connectivity mandates, and lowers egress data-transfer costs. It is **not encrypted by default** — add **MACsec** on supported ports, or run an **IPsec VPN over Direct Connect** (private IP VPN) for end-to-end encryption without public IPs. (AWS) (Stormit)
- **Site-to-Site VPN** is IPsec over the internet, fast to provision, cheap, with two tunnels across AZs by default — ideal as the **automatic BGP failover** for Direct Connect, or as the *starting* connectivity for a fast-growing company before committing to DX lead time and port fees. AWS best practice (Well-Architected Reliability pillar REL02-BP02) is a Site-to-Site VPN terminating on a **Transit Gateway** as backup for mission-critical workloads; BGP naturally prefers DX (shorter AS path), and VPN is used only on DX failure. Standard VPN tunnels cap at ~1.25 Gbps each; per AWS's November 2025 announcement ("AWS Site-to-Site VPN announces 5 Gbps bandwidth tunnels"), new **Large Bandwidth Tunnels** support "up to 5 Gbps bandwidth per tunnel, a 4x improvement from existing limit of 1.25 Gbps" — but only when terminating on Transit Gateway or Cloud WAN, not on a Virtual Private Gateway (VGW). Note that a single network flow still

cannot exceed 1.25 Gbps even with ECMP across multiple tunnels.

- **Decision criteria for Dyaboo:** start VPN-only if warehouse ↔ cloud traffic is modest; move to **DX primary + VPN failover** once WMS ↔ cloud sync throughput or compliance demands it (the per-flow 1.25 Gbps ceiling is a concrete trigger — a single large sync stream that needs more than that cannot be served by VPN regardless of tunnel count). Maximum resilience = two DX connections at two DX locations terminating on separate devices (99.99% SLA). (AWS)
- *Azure ExpressRoute* is the analogous private-circuit product if Dyaboo ever runs multi-cloud or adopts Entra/M365 heavily; it plays the same role as Direct Connect (dedicated private circuit, not over public internet). Not needed in an AWS-primary design.

VPC design & segmentation.

- Multi-AZ VPC with tiered subnets: **public** (ALB, NAT GW), **private-app** (.NET 8 PLM/SAG APIs), **data** (RDS PostgreSQL, ElastiCache), and **management** (Session Manager endpoints). Keep databases in private/data subnets with no internet route.
- **Isolate the public-facing PLM/SAG cloud tier from the on-prem WMS:** route on-prem ↔ cloud traffic only through the Transit Gateway/DX path, and use **Transit Gateway route tables as segmentation boundaries**

(separate prod/non-prod/DMZ domains so a compromised public component cannot reach the WMS or other VPCs laterally). (Simple AWS)

- **Security groups** (stateful, instance/ENI level) as the primary control; **NACLs** (stateless, subnet level) as coarse defense-in-depth. Default-deny, least-privilege rules referencing other SGs rather than CIDRs.
- **AWS Network Firewall** in a centralized **inspection VPC** behind the Transit Gateway for stateful IPS, domain allowlisting, and **egress control** (block data-exfiltration/malware callbacks, Suricata-compatible rules). Centralize rather than per-VPC to cut cost and management.
- **AWS PrivateLink / VPC endpoints** so traffic to AWS services (S3, Secrets Manager, ECR, etc.) stays on the AWS backbone with no internet gateway; gateway endpoints (S3/DynamoDB) are free, interface endpoints are hourly-priced. (Inventive HQ)

Zero-Trust networking (NIST SP 800-207 / 800-207A).

Shift from network-location trust to **identity-centric, per-request verification**. Concretely for Dyaboo: every service-to-service call authenticated (workload identity + mTLS), microsegmentation via SG/TGW route tables, a Policy Decision/Enforcement Point model, continuous telemetry, and "assume breach." NIST SP 800-207A specifically addresses enforcing granular application-level policy across

hybrid/multi-cloud using API gateways, sidecar proxies, and workload-identity frameworks (e.g., SPIFFE/SPIRE). Start with identity (MFA/PAM/JIT) as the first Zero-Trust anchor, then layer microsegmentation and mTLS. (NIST CSRC)

3. Data Security (At Rest & In Transit)

Encryption at rest & key management.

- **RDS for PostgreSQL / Aurora:** enable storage encryption at creation — it covers the instance, automated backups, read replicas, and snapshots transparently. **The KMS key cannot be changed after creation**, so choose a **customer-managed key (CMK)** up front (to migrate an unencrypted DB or change keys, snapshot → copy with new key → restore, or use DMS/logical replication with minimal downtime). (AWS)
- **AWS KMS with envelope encryption and automatic key rotation;** scope key policies with least privilege using the `(kms:ViaService)` condition (e.g., `(rds.<region>.amazonaws.com)`). Audit all key use via CloudTrail. Enforce with AWS Config rules `(rds-storage-encrypted)` and `(rds-snapshots-encrypted)`, and Security Hub controls. (AWS + 2)
- **CloudHSM** only if a compliance mandate requires single-tenant FIPS 140-2/3 Level 3 custody of keys;

otherwise KMS CMKs suffice and are far lower-overhead.

- **On-prem PostgreSQL (WMS):** enable filesystem/volume encryption (LUKS/BitLocker) plus PostgreSQL TDE-equivalent options; for portable, centralized key management across on-prem + cloud, **HashiCorp Vault** (Transit engine as encryption-as-a-service, dynamic DB credentials, PKI) is the strongest open-source option. Vault's **dynamic secrets** generate short-lived, auto-revoked PostgreSQL credentials per consumer — eliminating shared static DB passwords.

[Techplained](#)

- **Backup encryption:** all RDS snapshots inherit CMK encryption; encrypt on-prem WMS backups with the same Vault/KMS-managed keys and test restores.

Secrets management.

- **AWS-native: Secrets Manager** for AWS-centric workloads — native RDS rotation via Lambda, IAM-based access, CloudTrail audit. Per the AWS Secrets Manager pricing page, it costs "\$0.40 per secret per month and \$0.05 per 10,000 API calls" (identical across all 36 AWS regions; a customer-managed KMS key adds \$1.00/month per key). Parameter Store SecureString is a free option for config values. [Techplained](#) [Infisical](#)
- **Portable/multi-cloud: HashiCorp Vault** for dynamic secrets, cross-environment (on-prem WMS + AWS)

reach, and encryption-as-a-service. Decision rule: AWS-only → Secrets Manager; hybrid/dynamic-secrets/multi-cloud → Vault. Never store secrets in `.env`, code, or CI variables. (Qcecuring) (Gupta Deepak)

Encryption in transit.

- **TLS 1.2 minimum, TLS 1.3 preferred** for all external and internal traffic; HSTS on the Next.js/PLM/SAG front ends; require encrypted DB connections (`sslmode=require`)/(`verify-full`).
- **mTLS for service-to-service** (Zero-Trust east-west): issue workload certs from **AWS Private CA (ACM PCA)** with a proper root→intermediate hierarchy; for short-lived rotating certs use SPIFFE/SPIRE as an SDS provider, or a service mesh. On Kubernetes, **cert-manager** automates issuance/renewal (ACM PCA or Let's Encrypt issuers). For public endpoints, **ACM** provisions and auto-renews public TLS certs for ALB/CloudFront/API Gateway at no extra cost. (AWS + 2)
- **Securing on-prem PostgreSQL ↔ cloud .NET APIs:** keep this traffic on the DX/VPN private path, enforce TLS (and ideally mTLS) on the DB connection, restrict by SG/firewall to the specific app subnets, and pull DB credentials from Vault/Secrets Manager at runtime.

Field-level protection, masking, tokenization (financials, product-design IP, supplier/customer PII).

- **Column/field-level encryption** for the most sensitive fields (financials, design IP, PII) on top of at-rest encryption — [pgcrypto](#) symmetric encryption in PostgreSQL, or application-side encryption in the .NET layer with keys from KMS/Vault, so a DBA or a stolen snapshot alone cannot read them. ISO 27001:2022 A.8.11 (data masking) and A.5.34 (PII protection) expect field-level masking/pseudonymization rather than relying on disk encryption alone. [Xata](#) [GitHub](#)
 - **GDPR pseudonymization/tokenization** for supplier/customer personal data: tokenize direct identifiers (reversible via a separately secured key/vault per GDPR Art. 32), generalize quasi-identifiers, and use **PostgreSQL Anonymizer** or commercial field-level tools (e.g., Baffle) for masking in non-prod and analytics. Remember pseudonymized data is still personal data under GDPR and must be protected at the same level as the source. [Xata + 2](#)
 - **Data masking** in lower environments and role-based masking (RBAC) so only personnel with legitimate need see full values. [Baffle](#)
-

4. Application Security (AppSec) & DevSecOps

.NET 8 backend hardening (OWASP Top 10).

- **Token handling:** validate signature/issuer/audience/expiry locally via JWKS (above); never accept self-minted tokens; cache tokens server-side; rotate/persist refresh tokens securely.
- **Input validation & injection:** parameterized queries / EF Core (never string-concatenated SQL); validate and canonicalize all input; output-encode.
- **Anti-CSRF:** use ASP.NET Core's built-in antiforgery tokens (synchronizer pattern) for cookie-authenticated state-changing requests; mark session cookies HttpOnly. Note CSRF defenses are void under XSS — fix XSS first. (OWASP Cheat S...)
- **Secure headers:** add CSP (start in (Content-Security-Policy-Report-Only), avoid (unsafe-inline), move to nonces/hashes), (X-Content-Type-Options: nosniff), (X-Frame-Options: DENY), HSTS; remove (Server)/(X-Powered-By)/(X-AspNet-Version) fingerprinting headers. Place header middleware early. (ASP.NET Hacker) (ASP.NET Hacker)
- **Rate limiting:** use .NET 8's built-in rate-limiting middleware ((AddRateLimiter), (UseRateLimiter)) — Fixed/Sliding Window, Token Bucket, or Concurrency — partitioned per client/user, returning 429 with (Retry-After). (Note: .NET 8 made (AddRateLimiter) mandatory when the middleware is used.) (OneUptime + 2)

- **Secrets:** pull from Secrets Manager/Vault; never in `appsettings.json` committed to source.

Next.js frontend hardening.

- **XSS prevention:** rely on React's default escaping, avoid `dangerouslySetInnerHTML`, and enforce a strict CSP. Validate inputs with Zod/Yup at boundaries. `Digiq`
- **Secure cookie/token handling:** HttpOnly+Secure+SameSite session cookies, no tokens in localStorage.
- **SSR/SSRF:** govern outbound `fetch`/server-action requests with DNS allowlists, block link-local/metadata endpoints (169.254.169.254) via a central fetch wrapper to prevent SSRF pivoting into cloud metadata. `Digiq`
- **Dependency security:** lockfile integrity, `npm audit`/Dependabot, and SCA in CI (below).

WAF for public PLM/SAG. Front CloudFront/ALB/API Gateway with AWS WAF using managed rule groups: the **Core Rule Set (AWSManagedRulesCommonRuleSet)** for OWASP-Top-10-class protection — its capacity is exactly **700 WCU** (per the AWS CLI `describe-managed-rule-group` output), and an AWS WAF web ACL's base price includes up to 1,500 WCUs, leaving ample room for additional groups. Add **Known Bad Inputs**, **Amazon IP Reputation List**, and **SQLi/admin-protection** sets, plus custom **rate-based rules**. Deploy new rule groups in **Count mode for 1-2 weeks**, review

sampled requests/CloudWatch metrics for false positives, then switch to Block; use rule-action overrides for noisy rules. Manage WAF org-wide with **AWS Firewall Manager**. Open-source/portable alternative: ModSecurity with the **OWASP Core Rule Set**.

CI/CD security gates (tools suited to .NET + TypeScript).

- **SAST: GitHub Advanced Security / CodeQL** (highest accuracy on supported languages, native to GitHub PRs) — best fit if Dyaboo is on GitHub; alternatives **Semgrep** (fast, custom rules, strong on TS/JS/C#) and **SonarQube** (quality + security gate). (NomadX)
- **SCA / dependencies: Dependabot** (auto-PRs) + **Trivy** or **Snyk** (NuGet + npm CVE coverage; Trivy is free/CLI-first, Snyk has deeper remediation/IDE UX). OWASP Dependency-Check as a free CI option. (OX Security)
(Aikido)
- **Secrets scanning: Gitleaks / TruffleHog** at commit and history; enable **push protection**. (CyberDefenders)
- **Container/image scanning: Trivy** (OS + language deps, misconfig, secrets, SBOM) or **Grype**; scan ECR images.
(AWARE7 GmbH)
- **IaC scanning: Checkov**
(Terraform/CloudFormation/K8s), tfsec, or KICS.
(CyberDefenders)
- **DAST: OWASP ZAP** against staged PLM/SAG.

- **Practical sequencing:** Trivy + Dependabot covers most SCA needs at zero license cost; add Snyk/Checkmarx only if compliance accountability requires a commercial vendor. Deploy gates one pipeline/one category at a time, define blocking-severity SLAs first, and tune noise to keep developer trust. (CyberDefenders)

DevSecOps for both on-prem and cloud targets + supply chain.

- One pipeline (GitHub Actions or GitLab CI) building artifacts once, deploying to **both** the on-prem WMS and the cloud PLM/SAG via environment-specific stages with separate, least-privilege deploy credentials (OIDC-federated to AWS for keyless cloud deploys; scoped service account for on-prem).
- **Supply chain:** generate an **SBOM** (Syft, CycloneDX/SPDX) every build; **sign images and SBOM attestations with Cosign/Sigstore keyless** (Fulcio short-lived certs + Rekor transparency log, via GitHub Actions OIDC — no long-lived keys). Enforce "**no signature/SBOM, no deploy**" with a Kubernetes admission controller (Kyverno/OPA Gatekeeper) or AWS-side verification. Use **CodeArtifact** (or Artifactory/Nexus) with package-origin controls to block dependency-confusion. Track SBOMs centrally (OWASP Dependency-Track) so a new CVE query returns affected artifacts in seconds. (Systemsarchite...)

5. Monitoring, Logging & Incident Response

Hybrid centralization is the core problem. Security Hub/GuardDuty give excellent AWS-side coverage but are **blind to on-prem servers, endpoints, and the warehouse WMS.** The recommended pattern: [Red-team](#)

- **AWS-native layer: CloudTrail** (API audit, with log-file validation + KMS encryption, org trail to the security/log-archive account), **CloudWatch** (metrics/logs/alarms), **VPC Flow Logs**, **GuardDuty** (threat detection across EC2/S3/RDS/IAM/EKS), **Security Hub** (CSPM, compliance-standard scoring, aggregates GuardDuty/Inspector/Macie), **Amazon Detective** (investigation/graphing), **Amazon Macie** (PII discovery in S3), and **Inspector** (vuln scanning of EC2/ECR/Lambda). [DEV Community](#) [Red-team](#)
- **Cross-platform SIEM** for true hybrid visibility:
 - **Open-source/portable: Wazuh** — agent-based endpoint/file-integrity monitoring on on-prem WMS + cloud instances, with built-in modules ingesting CloudTrail, GuardDuty, Macie, VPC Flow; built-in PCI/HIPAA/GDPR/NIST compliance dashboards; MITRE-mapped rules. Forward Security Hub/GuardDuty findings into Wazuh for one unified view. It is substantially cheaper than

equivalent commercial stacks, but you own the operational burden (patching, scaling, HA).

[Simplico](#) [Red-team](#)

- **AWS-hosted open SIEM:** the **SIEM on Amazon OpenSearch Service** solution (aws-samples) with ~2,200 Sigma rules.
- **Enterprise: Splunk** (broadest connectors, UEBA/ML) or **Microsoft Sentinel** (best if Dyaboo adopts Entra/M365). Elastic/ELK and Grafana Loki are portable log-aggregation options.
- **Decision rule for a mid-size company:** run **AWS-native detection + Wazuh** as the hybrid correlator first; graduate to Splunk/Sentinel when SOC maturity, UEBA, or connector breadth justify the cost.

Log aggregation, anomaly detection, alerting. Ship on-prem WMS + cloud logs to the central SIEM and a write-once log-archive account (S3 with Object Lock for tamper-evidence). Alert on high-value events: root-account use, IAM changes, security-group changes, failed logins/brute force, new IAM principals, KMS key disable, and high-severity vulns. Tune thresholds to control noise; assign an owner and response-time target per alert category. [Konfirmity](#) [Konfirmity](#)

Incident response & compliance.

- Build **runbooks** (the *how*: trigger, steps, systems, evidence to capture, sign-off) distinct from **playbooks**

(the *response frame*), covering the full lifecycle: detection → triage → containment → eradication → recovery → lessons learned. Start from the **AWS Incident Response Playbook samples** (aws-samples). Auditors compare runbook steps to actual artifacts (tickets, CloudTrail exports, sign-offs), so make **evidence capture a runbook step** (e.g., "export relevant CloudTrail entries to the incident folder before remediation") to avoid the "evidence trap" of rotated logs. (AxiPro + 2)

- **SOC 2 / ISO 27001 evidence:** CloudTrail/Config/Security Hub snapshots, KMS key policies, IAM least-privilege reviews, change tickets, and a Logging-and-Monitoring control map (log sources, retention, alert types, owners). **AWS Audit Manager** auto-collects evidence against ISO 27001/SOC 2 frameworks; **AWS Artifact** supplies AWS's own SOC/ISO/PCI reports for vendor due diligence. Run **tabletop exercises and DR/failover tests at least annually** and retain records. (Konfirmity + 2)
- **GDPR:** maintain a data-processing inventory for supplier/customer PII, support data-subject requests (access/erasure endpoints), pseudonymize where feasible, and document lawful basis — SOC 2's privacy criterion overlaps but does not replace GDPR obligations. (Konfirmity)

Recommendations

Phase 1 — Foundation (0-3 months): close the highest-risk gaps.

1. Stand up the AWS Organization (prod / non-prod / security-log-archive / shared-networking accounts); enable CloudTrail org trail, Config, GuardDuty, Security Hub everywhere.
2. Federate on-prem AD to **IAM Identity Center** (direct AD Connector or SAML+SCIM); enforce **phishing-resistant MFA**; eliminate IAM users; adopt **Session Manager** for all admin access (close port 22/3389).
3. Establish connectivity: **Site-to-Site VPN (2 tunnels, BGP, on Transit Gateway)** now; plan Direct Connect. Build the tiered VPC, isolate WMS path, default-deny SGs.
4. Turn on **RDS PostgreSQL encryption with a customer-managed KMS CMK** (decide before creation), enforce TLS on all DB connections, and encrypt on-prem WMS volumes/backups.
5. Front PLM/SAG with **AWS WAF** (Core Rule Set + IP reputation, Count mode first).
6. Wire **.NET 8 JWT bearer validation** and **Auth.js OIDC** with HttpOnly cookies; add secure headers, antiforgery, and rate limiting.

Phase 2 — Harden (3-6 months).

1. Implement the **DevSecOps pipeline gates** (CodeQL/Semgrep SAST, Dependabot+Trivy SCA, Gitleaks secrets, Checkov IaC, ZAP DAST) with defined blocking-severity SLAs; start **SBOM + Cosign signing**.
2. Deploy **secrets management** (Secrets Manager for AWS, Vault if hybrid/dynamic DB creds) and migrate all secrets out of config.
3. Stand up the **hybrid SIEM** (AWS-native + Wazuh), forward findings, define alert runbooks, and ship on-prem WMS logs centrally.
4. Move WAF rules to **Block**; add Network Firewall egress control in a central inspection VPC.
5. Add **column/field-level encryption** for financials/IP/PII and **GDPR pseudonymization** in non-prod.

Phase 3 — Mature (6-12+ months).

1. **Direct Connect primary + VPN failover**; consider dual-DX for 99.99% SLA as traffic grows.
2. **Zero-Trust microsegmentation** (TGW route-table domains), **mTLS** for service-to-service via ACM Private CA / SPIFFE, and workload identity.
3. **JIT privileged access** (TEAM/Change Manager or Entra PIM) and, if needed, enterprise PAM (CyberArk/WALLIX).

4. **Continuous compliance:** AWS Audit Manager evidence automation, annual tabletop + DR tests, and SOC 2 Type II / ISO 27001 certification readiness.

Benchmarks that change the plan: move VPN→Direct Connect when a single WMS ↔ cloud sync flow needs more than the 1.25 Gbps per-flow VPN ceiling, or when latency/jitter harms warehouse operations; adopt commercial SIEM (Splunk/Sentinel) when alert volume/SOC headcount or UEBA needs exceed Wazuh; adopt enterprise PAM when third-party vendor access or session-recording mandates appear; move from Cognito to Keycloak (or vice-versa) based on whether on-prem AD federation depth or AWS operational simplicity dominates.

Caveats

- **"SAG" is not expanded in the brief.** I treated it as a public-facing cloud ERP module with the same control profile as PLM; validate its actual data sensitivity (if it handles financials, apply the §3 field-level controls).
- **Cost, latency, and scalability figures should be re-validated against current AWS pricing and your own load tests before budgeting.** The Secrets Manager (\$0.40/secret/mo) and WAF (700 WCU CRS) figures are confirmed against AWS's own pricing/CLI output as of mid-2026; the Keycloak ~12,000 req/s figure is from Keycloak's own Oct-2025 benchmark on a very large

Aurora instance and will not reflect a modest deployment; the Wazuh cost advantage is directional, not a quoted price.

- **AWS recommends IAM Identity Center for new deployments**, but if Dyaboo has heavy existing ADFS investment, SAML federation remains valid; the choice affects MFA and attribute-mapping options.
- Some cited sources are dated 2026 and reflect current product states (e.g., Security Hub's re:Invent 2025 unification of GuardDuty/Inspector/Macie, and the Nov-2025 5 Gbps VPN tunnels); confirm exact feature availability in your region at implementation time.
- This strategy assumes Dyaboo can resource the operational overhead of any open-source choices (Keycloak, Vault, Wazuh); if not, the AWS-native managed equivalents are the safer default despite some lock-in.
- Regulatory specifics (GDPR data-residency, any sector rules for textile/apparel supply-chain data) should be confirmed with counsel; this is a security-architecture strategy, not legal advice.